

Object-Oriented Programming Concepts in C++

Abstraction, Dynamic Lookup, Subtyping & Inheritance

Course: Object-Oriented Programming

Submitted To: Miss Aliya Farooq

Submitted By: Muhammad Uzair — 2026(S)-CS-96

1. Introduction

Object-Oriented Programming (OOP) is a programming paradigm centered on the concept of objects — self-contained units that combine data (attributes) and behavior (methods). C++ is one of the most widely used object-oriented languages, offering a powerful and flexible framework for applying OOP principles in systems programming, game development, embedded systems, and enterprise software.

The four pillars that define and distinguish OOP are:

- Abstraction — hiding implementation complexity behind a simplified interface
- Dynamic Lookup — resolving method calls at runtime based on the actual object type
- Subtyping — allowing a derived type to be used wherever a base type is expected
- Inheritance — enabling a class to acquire properties and behaviors of another class

This assignment explores each of these concepts in depth, demonstrates how C++ implements them using its language features, and provides illustrative code examples to reinforce understanding.

2. Abstraction

2.1 Definition and Purpose

Abstraction is the process of exposing only the essential details of an object while hiding the underlying complexity. It is a cognitive tool that allows programmers to work with high-level models without needing to understand every implementation detail. In C++, abstraction is achieved through the use of classes, access specifiers (public, private, protected), and abstract classes (pure virtual functions).

The primary goal of abstraction is to reduce complexity and increase efficiency. A user of a class should only need to know what a function does, not how it does it. This separation of interface from implementation is foundational to building large, maintainable software systems.

2.2 Abstraction via Classes and Access Specifiers

In C++, a class encapsulates data members and member functions. By declaring internal data as private, the class ensures that only its own methods can manipulate that data. Public methods form the interface through which external code interacts with the object.

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    double balance;    // Hidden from external access
```

```

string accountNumber;

void logTransaction(double amount) { // Internal helper
    cout << "Transaction: " << amount << endl;
}

public:
    BankAccount(string accNo, double initialBalance)
        : accountNumber(accNo), balance(initialBalance) {}

    void deposit(double amount) { // Public interface
        if (amount > 0) {
            balance += amount;
            logTransaction(amount);
        }
    }

    double getBalance() const { // Read-only access
        return balance;
    }
};

int main() {
    BankAccount acc("PK-001", 5000.0);
    acc.deposit(1500.0);
    cout << "Balance: " << acc.getBalance() << endl;
    return 0;
}

```

In this example, the user of `BankAccount` only needs to call `deposit()` and `getBalance()`. They have no access to the `balance` variable directly or to the internal `logTransaction()` helper. This is abstraction in practice.

2.3 Abstract Classes and Pure Virtual Functions

C++ also supports abstract classes — classes that define an interface but leave the implementation to derived classes. An abstract class contains at least one pure virtual function (declared with `= 0`). It cannot be instantiated directly, ensuring that only concrete derived classes are used.

```

class Shape {
public:
    virtual double area() = 0; // Pure virtual — must be overridden
    virtual void draw() = 0;
    virtual ~Shape() {}
};

class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
}

```

```
double area() override { return 3.14159 * radius * radius; }
void draw() override { cout << "Drawing Circle" << endl; }
};
```

Abstract classes enforce a contract: any class inheriting from Shape must implement area() and draw(). This guarantees consistent behavior across all shape types, regardless of how each shape computes its area internally.

3. Dynamic Lookup

3.1 Definition

Dynamic lookup, also known as dynamic dispatch or runtime polymorphism, is the mechanism by which a program determines at runtime which method implementation to call, based on the actual type of the object rather than the declared type. This is a cornerstone of polymorphic behavior in OOP.

In C++, dynamic lookup is achieved through virtual functions and the virtual table (vtable) mechanism. When a class declares a virtual function, the compiler generates a vtable — a table of function pointers — for that class. Each object of the class holds a hidden pointer (vptr) to its class's vtable. When a virtual method is called through a base class pointer or reference, the runtime looks up the actual function address through this table.

3.2 Virtual Functions in C++

To enable dynamic lookup in C++, a function must be declared with the virtual keyword in the base class. Without this keyword, function calls are resolved at compile time (static dispatch), and the base class version is always called regardless of the actual object type.

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() { // Virtual enables dynamic lookup
        cout << "Some generic animal sound" << endl;
    }
    virtual ~Animal() {}      // Virtual destructor is important
};

class Dog : public Animal {
public:
    void makeSound() override {
        cout << "Woof! Woof!" << endl;
    }
};

class Cat : public Animal {
```

```

public:
    void makeSound() override {
        cout << "Meow!" << endl;
    }
};

int main() {
    Animal* animals[3];
    animals[0] = new Animal();
    animals[1] = new Dog();
    animals[2] = new Cat();

    for (int i = 0; i < 3; i++) {
        animals[i]->makeSound(); // Resolved at runtime!
    }

    for (int i = 0; i < 3; i++) delete animals[i];
    return 0;
}

```

Output of the above program:

```

Some generic animal sound
Woof! Woof!
Meow!

```

Even though all calls are made through a pointer of type `Animal*`, the runtime correctly dispatches to `Dog::makeSound()` and `Cat::makeSound()`. This is the power of dynamic lookup — the correct method is selected based on the actual object, not the pointer type.

3.3 The vtable Mechanism

Internally, C++ compilers implement dynamic dispatch using virtual tables. Each class with virtual functions has a vtable — a static array of function pointers. Each object of such a class contains a hidden `vp`tr (virtual pointer) that points to the vtable of its actual class. When a virtual method is called, the runtime follows the `vp`tr, looks up the function in the vtable, and jumps to it. This adds a small but constant overhead compared to direct function calls, which is the tradeoff for runtime flexibility.

4. Subtyping

4.1 Definition

Subtyping is a relationship between types where a subtype can be substituted for a supertype without affecting the correctness of the program. This principle, formalized as the Liskov Substitution Principle (LSP), states: if `S` is a subtype of `T`, then objects of type `T` may be replaced with objects of type `S` without altering any of the desirable properties of the program.

In C++, subtyping is implemented through public inheritance. When a class `Derived` publicly inherits from `Base`, `Derived` is a subtype of `Base`. A pointer or reference to `Base` can legally refer to a `Derived` object. This enables writing generic code that operates on the base type, while working correctly with any subtype at runtime.

4.2 Subtyping in C++

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    virtual void startEngine() {
        cout << "Engine started." << endl;
    }
    virtual void describe() {
        cout << "I am a Vehicle." << endl;
    }
    virtual ~Vehicle() {}
};

class Car : public Vehicle {
public:
    void describe() override {
        cout << "I am a Car." << endl;
    }
};

class Truck : public Vehicle {
public:
    void describe() override {
        cout << "I am a Truck." << endl;
    }
};

// Function accepts base type — works with any subtype
void processVehicle(Vehicle& v) {
    v.startEngine();
    v.describe();
}

int main() {
    Car myCar;
    Truck myTruck;

    processVehicle(myCar);    // Car substitutes Vehicle
    processVehicle(myTruck);  // Truck substitutes Vehicle
    return 0;
}
```

The function `processVehicle()` is written to accept a `Vehicle` reference, but it works seamlessly with `Car` and `Truck` objects. This is the subtyping relationship at work. The caller does not need to know the concrete type — any subtype satisfies the contract.

4.3 The Liskov Substitution Principle (LSP)

LSP requires that a subtype must honor all guarantees provided by the supertype. For example, if `Vehicle` guarantees that `startEngine()` always outputs a message, then `Car` and `Truck` must also honor this guarantee. Violating LSP leads to fragile code — functions expecting a base type will break when given a subtype that behaves differently than expected.

C++ enforces syntactic subtyping through public inheritance, but it is the programmer's responsibility to ensure semantic compliance with LSP. A well-designed class hierarchy respects the behavioral contracts of its base classes.

5. Inheritance

5.1 Definition and Purpose

Inheritance is a mechanism in which a new class (derived class or child class) acquires the properties and behaviors of an existing class (base class or parent class). It promotes code reuse, establishes natural hierarchies, and enables the extension of existing functionality without modifying the original class.

C++ supports several forms of inheritance: single inheritance (one base class), multiple inheritance (multiple base classes), multilevel inheritance (chain of inheritance), hierarchical inheritance (multiple derived classes from one base), and hybrid inheritance (a combination of the above).

5.2 Single Inheritance

```
#include <iostream>
using namespace std;

class Person {
protected:
    string name;
    int age;

public:
    Person(string n, int a) : name(n), age(a) {}

    void introduce() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

class Student : public Person {
```

```

private:
    string rollNumber;
    double gpa;

public:
    Student(string n, int a, string roll, double g)
        : Person(n, a), rollNumber(roll), gpa(g) {}

    void display() {
        introduce();    // Inherited from Person
        cout << "Roll: " << rollNumber << ", GPA: " << gpa << endl;
    }
};

int main() {
    Student s("Ali", 20, "CS-301", 3.85);
    s.display();
    return 0;
}

```

Student inherits name, age, and introduce() from Person. It adds its own attributes (rollNumber, gpa) and behavior (display()). This avoids duplication — the Person class can also be the base for Teacher, Employee, or any other person-like entity.

5.3 Multiple Inheritance

C++ uniquely supports multiple inheritance, allowing a class to derive from more than one base class. While powerful, it must be used carefully to avoid the diamond problem — an ambiguity that arises when two parent classes share a common ancestor.

```

class Flyable {
public:
    void fly() { cout << "Flying..." << endl; }
};

class Swimmable {
public:
    void swim() { cout << "Swimming..." << endl; }
};

class Duck : public Flyable, public Swimmable {
public:
    void quack() { cout << "Quack!" << endl; }
};

int main() {
    Duck d;
    d.fly();    // Inherited from Flyable
    d.swim();   // Inherited from Swimmable
    d.quack();  // Duck's own method
    return 0;
}

```



```
}
```

5.4 Constructor Chaining in Inheritance

When a derived class object is created, the base class constructor is called first. C++ uses constructor initialization lists to pass arguments to the base class constructor. This ensures that all inherited data members are properly initialized before the derived class constructor body executes.

```
class Engine {
public:
    int horsepower;
    Engine(int hp) : horsepower(hp) {
        cout << "Engine created with " << hp << " HP" << endl;
    }
};

class Car : public Engine {
public:
    string model;
    Car(string m, int hp) : Engine(hp), model(m) { // Base ctor called first
        cout << "Car model: " << model << " is ready." << endl;
    }
};

int main() {
    Car c("Civic", 180);
    return 0;
}
```

6. Interplay Between the Four Concepts

Although each concept has its own distinct definition, in practice they work together to form a coherent object-oriented design. The table below summarizes how they relate:

Concept	Role in OOP	C++ Mechanism
Abstraction	Hides complexity, exposes only what is necessary	Classes, access specifiers, abstract classes
Dynamic Lookup	Selects correct method at runtime	Virtual functions, vtable, vptr
Subtyping	Allows subtypes to replace base types	Public inheritance, base class pointers/references
Inheritance	Reuses and extends existing class features	Derived classes, constructor chaining, overriding

A well-engineered C++ program leverages all four of these concepts simultaneously. For example, a graphics framework might define an abstract Shape class (abstraction), use virtual draw() methods (dynamic lookup), store all shapes as Shape* pointers (subtyping), and define Circle, Rectangle, and Triangle by inheriting from Shape (inheritance).

7. Practical Applications

Object-Oriented Programming is the standard for modern software because it allows developers to model real-world systems efficiently. Some common applications include:

- **GUI Development:** Modern operating systems (Windows, macOS) use OOP to represent interface elements like buttons, windows, and menus as objects that can be reused and customized.
- **Game Development:** In games, every character, weapon, and environment item is treated as an individual object with its own health, position, and actions.
- **Banking & Finance:** OOP is used to manage customer accounts and transactions securely. By using abstraction, sensitive data like balances can be hidden and only modified through specific, secure methods.
- **Simulations and Modeling:** Scientists use OOP to simulate complex systems, such as weather patterns or flight simulators, where different physical variables interact as independent objects.
- **Database Management:** Modern databases often use object-oriented structures to organize data into clear categories, making it easier to search, update, and maintain large amounts of information.